Lecture 05.02: Classes & Objects

📌 Executive Summary

Classes as blueprints, objects as runtime heap instances, identity vs equality, memory allocation, and object introspection.

🔬 Theoretical Foundations & Principles

1. Blueprint vs Instance

A **Class** is a user-defined prototype or type definition specifying attributes and methods. An **Object** is a concrete instance allocated in CPython heap memory:

$$\text{Instance} \sim \mathcal{C}(\mathbf{w})$$

2. Memory Anatomy & Identity

Every Python object possesses three core invariants:

- **Identity:** Its unique memory address in CPython (`id(obj)`).
- **Type:** The class from which it was instantiated (`type(obj)`).
- **Value:** The data stored in its attributes (`obj.__dict__`).

Equality operator `==` checks value equivalence (`__eq__`), while `is` tests memory identity (`id(a) == id(b)`).

💻 Production Implementation & Code

```
class Student:
    pass

s1 = Student()
s2 = Student()

print(f"s1 id: {hex(id(s1))}")
```

```
print(f"s2 id: {hex(id(s2))}")
print(f"s1 is s2? {s1 is s2}")  # False: Distinct heap objects
```

⚙️ Algorithmic Complexity & Systems Analysis

- **Time Complexity:** Method dispatch overhead is $\mathcal{O}(1)$ via CPython descriptor resolution.
- **Auxiliary Space:** Instance attribute memory allocation in `__dict__` vs `__slots__`.